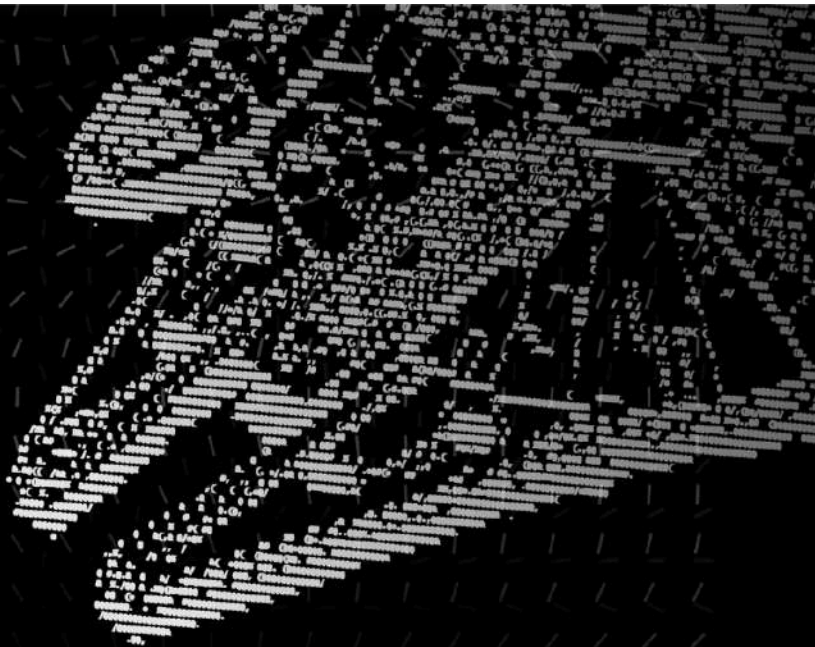




E-Cell
JSS NOIDA



STARFORGE

HACKATHON 2026

Total Prize Pool

₹80,000

.pathway

Title
Partner



weya.ai

Platinum
Partner

rime

Voice Models
Partner



drant

Vector Search
Partner



10th-13th August, 2026
Online - Offline Mode



JSS University
C-20/1, Sector 62, Noida

REGISTRATIONS LIVE!

Contact :-

Aman Agrahari: +91 7234909407

Prateek Yadav: +91 9161580148

INCOMING TRANSMISSION

Two missions. One forge.

Build the kind of AI products teams need now, or explore the architecture that may shape what comes next.

Co-presented by Pathway (pathway.com). Organized by JSS University and E-Cell JSS.

StarForge gives you two ways to enter the frontier. The first is a practical product mission around voice experiences and voice agents. The second is a research mission around Pathway's Post-Transformer reasoning model architecture, Dragon Hatchling (BDH).

Neither track assumes that you already know the sponsor technologies. The problem statement teaches the minimum you need, gives you realistic scopes for a 10 hour sprint and a 5 day mission, and shows what a strong GitHub showcase should contain.

MISSION	VOXFORGE	DRAGONFORGE
BUILD	A useful voice experience or agent where Rime speech is essential, Qdrant adds meaningful intelligence, and the flow survives real user behavior.	A builder contribution around the research questions BDH is opening: explain, evaluate, reproduce, instrument, or investigate.
CHOOSE IT IF	You want product engineering, voice, agents, retrieval, evaluation, workflows, or user experience.	You are already into deep learning, or you are ready to learn model internals, experiments, visualization, systems, or research.
	THE NORTH STAR Do not build the largest project. Build the clearest proof that your project works, matters, and can be trusted.	

00

Mission rules

These apply to both tracks

CHOOSE YOUR LAUNCH WINDOW

10 HOUR SPRINT

One complete flow or one verified insight.
Aim for clarity, reliability, and a memorable proof.

5 DAY MISSION

Add evaluation, controls, persistent behavior, reproducibility, and a reusable community artifact.

REQUIREMENT	WHAT GOOD LOOKS LIKE
PUBLIC GITHUB	Your repository must contain working code, setup instructions, an architecture diagram, screenshots or charts, a demo link, team contributions, and known limitations.
WORKING PROOF	Show one live behavior, one reproducible experiment, or one test that supports your central claim.
SHOWCASE CLIP	Include one 20 to 30 second clip or GIF that communicates the strongest moment without a long explanation.
AI ASSISTED CODING	AI tools are welcome. Judges may ask you to explain the code behind your main claim, change one input, and show the test that catches a wrong implementation.
SECRETS AND DATA	Keep API keys on the server. Use synthetic or deidentified information for health, finance, identity, and other sensitive workflows.

Use the launch window announced by the organizers. The same mission can be scoped as a focused sprint or expanded into a deeper build.



TRACK 01

VoxForge

Build a voice experience worth listening to.

Build a working product or prototype in which Rime generated speech drives the main interaction. Use Coda or Mist v3, keep every credential on the server, and demonstrate a real end to end flow.

Speech must be essential to the experience. If the same product works just as well as text on a screen, voice is not doing enough. The voice should respond quickly, suit the situation, handle at least one difficult production problem, and behave predictably during the demo.

StarForge adds two forces that can turn a good voice demo into a strong voice product or agent. Qdrant should give the system useful knowledge, memory, routing, or evaluation. Weya AI mentors can help you pressure test the workflow around tools, confirmations, transfers, campaigns, webhooks, analytics, and continuity across channels.

NOT A CHATBOT WITH A MICROPHONE

The user should benefit from speaking, listening, interrupting, or continuing a task without staring at a display. Build the interaction around what voice makes possible.

The required stack

REQUIREMENT	WHAT GOOD LOOKS LIKE
RIME	Use Coda or Mist v3 for spoken output. Choose the model, voice, language, and transport for the user and setting. Rime speech must be essential to the main flow.
QDRANT	Give Qdrant a real job. Use it for knowledge retrieval, user memory, similar case search, personalization, routing, or evaluation. Use payload filters to keep users and workflows isolated.
APPLICATION LAYER	Use any suitable service for speech recognition, language model reasoning, orchestration, transport, tool use, interface design, and evaluation.
WEYA AI LENS	Weya AI is not a required dependency since dev-tool access may not be available to you as a student. Use its mentors and resources to improve your product vision, phone and WhatsApp flows, multilingual behavior, tool calls, CRM or API integration, transfers, post call workflows, campaigns, webhooks, analytics, and multichannel continuity.

For most new Rime applications, Coda is the natural starting point. Mist v3 is worth testing when the fastest first audio is central to the product. In every case, validate the exact voice, language, endpoint, and transport used in the final demo.

Weya AI has a growing presence in the Indian BFSI sector. Its developer tools are not directly available to participants as a required stack today, so use Weya primarily as a mentor lens for production agent design and follow its work for future developer releases.

How the system can fit together



A strong architecture separates four kinds of context: the knowledge the agent may need, memories about this user, similar past cases, and the current task state. Qdrant can hold one or more of these. Payload fields can identify the user, workflow, language, risk level, timestamp, and memory type so the agent retrieves the right context and does not leak another user's information.

Reference GitHub repo to inspect

github.com/ankit1khare/rime-voice-agent

Use this as scaffolding, not as the target. Understand how the working voice agent handles transport, turn taking, barge in, and Rime speech. Then aim to build something more original, more measurable, and more production ready, with a meaningful Qdrant role and stronger failure handling.

01

Choose your voice mission

The eight routes from the Rime challenge brief

REAL TIME VOICE AGENTS

Build a conversation that stays coherent through turns, interruptions, recovery, and reconnects. Measure the full delay that the user experiences, not only model generation time.

VOICE FOR HIGH TRUST WORKFLOWS

Build for healthcare, insurance, finance, operations, public services, or another consequential task. Match confirmations, recovery, and human escalation to the risk. Use synthetic or deidentified data.

TOOLS FOR VOICE BUILDERS

Create a tool that helps teams compare prompts or voices, test pronunciation, catch regressions, measure latency, or evaluate what users hear. Qdrant can store test cases, ratings, and similar failures.

MULTILINGUAL VOICE EXPERIENCES

Build for one or more supported languages. Test real names, numbers, domain vocabulary, and code switching. Choose and route voices deliberately.

VOICE BEYOND THE SCREEN

Create a hands free experience for accessibility, mobility, hardware, laboratories, field work, or any setting where a screen is impractical.

LEARNING OUT LOUD

Build a tutor, coach, interview partner, or practice tool that teaches through short spoken turns and useful feedback. Use a separate evaluator when scoring the user's speech.

VOICES WITH CHARACTER

Build an interactive story, game, media experience, or character whose voice truly fits the setting. Shape delivery through voice choice, wording, and punctuation. Keep the result consistent.

WILDCARD

Build beyond the listed routes. Rime speech must still solve a clear interaction problem, and Qdrant must still make a meaningful contribution.

Go deeper: engineering moves that make a Rime project stand out

These are not extra requirements. Pick one engineering depth lever for a 10 hour build. For a 5 day build, combine two or three. The goal is to solve one hard production problem convincingly rather than sprinkle many features across the demo.

RIME ROUTE	ENGINEERING DEPTH THAT CAN MAKE IT MEMORABLE
REAL TIME VOICE AGENTS	Track the full latency budget. Support barge in. Record what audio the user actually heard. Recover from reconnects without duplicating or skipping the task state. Qdrant can retrieve only the unresolved task or relevant prior preference.
HIGH TRUST WORKFLOWS	Design confirmation levels around risk. Separate suggestions from irreversible actions. Add a safe human handoff and an auditable outcome. Qdrant can retrieve the right policy or similar resolved case with strict metadata filters.
TOOLS FOR VOICE BUILDERS	Create a reusable test corpus. Compare voices, prompts, pronunciation, latency, interruptions, and regressions. Store failures and human ratings in Qdrant, then retrieve similar failures when a new version breaks.
MULTILINGUAL VOICE	Route language and voice deliberately. Test code switching, names, numbers, addresses, and local domain terms. Use Qdrant payloads to keep knowledge, examples, and evaluation cases language aware.
VOICE BEYOND THE SCREEN	Design for the physical context: noisy rooms, hands busy work, accessibility, or limited display time. Keep turns short, make confirmations audible, and store only the observations or procedures the user may need again.
LEARNING OUT LOUD	Use short teaching turns, hints before answers, and explicit feedback loops. If you score pronunciation or speaking quality, use a separate evaluator. Qdrant can support spaced repetition, mistake memory, and personalized practice selection.
VOICES WITH CHARACTER	Keep persona, lore, and speaking style consistent across sessions. Test how wording and punctuation shape delivery. Use Qdrant for bounded character memory or world knowledge without letting old context derail the current scene.
WILDCARD	State the interaction problem first. Show why voice is essential. Give Qdrant one job that changes the product outcome. Then choose one hard voice problem and measure or demonstrate it clearly.
	DEPTH BEATS FEATURE COUNT A project becomes showcase worthy when one difficult detail is obviously solved: clean interruption recovery, a measurable latency win, a multilingual edge case, a safe handoff, a regression caught before release, or memory that is both useful and correctable.

StarForge Special Mission: Never Start From Zero

A particularly strong intersection of Rime, Qdrant, and the Weya AI mentor lens is a voice agent that never starts from zero. It remembers the right part of an unfinished task, retrieves only what matters now, completes an action or safe handoff, and recovers when the user interrupts, corrects it, returns later, or changes channel.

A NOTE ON CONTINUAL LEARNING

There are two different ideas that are easy to mix up. Memory files, vector stores such as Qdrant, summaries, RAG, and agent state are application memory outside the model. They are useful and are exactly what VoxForge teams can build with today. Model internal continual learning is different: the model itself would durably absorb experience and become better over time. Pathway is researching that direction with BDH, but the public BDH repository in this hackathon does not provide a continual learning system.

A SHOWCASE READY DEMONSTRATION CAN CONTAIN FIVE MOMENTS

1. The user explains a goal, preference, problem, or unresolved task.
2. Qdrant retrieves the right knowledge, memory, similar case, or evaluation record.
3. Rime speaks a response whose voice and delivery fit the situation.
4. The product completes a visible action, prepares a safe handoff, or produces a repeatable developer insight.
5. An interruption, correction, later session, or channel change proves that the system can recover without carrying the wrong memory forward.

A SOCIAL READY DEMO

Capture one unforgettable proof. It may be the interruption recovery, the cross session memory, the tool action, the multilingual switch, or the regression that your developer tool catches. A viewer should understand why the product matters in under 30 seconds.

How Qdrant can earn its place

REQUIREMENT	WHAT GOOD LOOKS LIKE
KNOWLEDGE	Retrieve policies, manuals, catalog entries, procedures, or other source material. Hybrid retrieval can combine semantic intent with exact names, codes, and identifiers.
USER MEMORY	Store useful preferences, corrections, unresolved tasks, and outcomes. Add a retention rule and a clear way to correct or delete wrong memories.
CASE MEMORY	Find similar past calls, successful resolutions, edge cases, or failures. This is especially useful for support, operations, collections, and developer tools.
ROUTING	Use metadata such as user, language, product, risk, geography, or workflow stage to retrieve only the context permitted for the current interaction.

REQUIREMENT	WHAT GOOD LOOKS LIKE
EVALUATION	Store test prompts, transcripts, audio references, human ratings, latency measurements, failure tags, and regression history so teams can find recurring problems.
	THE WEYA AI MENTOR LENS Bring mentors to your conversation state machine, not only your prompt. Ask where the agent should speak, confirm, act, retry, transfer, stop, and what information must survive after the call. Weya AI works across agents, campaigns, tools, webhooks, analytics, phone, video, and WhatsApp, which makes its mentors especially useful for production workflow design.

02

What judges should experience

The product proof, not a slide deck

- A specific user and a problem that is worth solving.
- A clear reason that voice is better than typing for this situation.
- A working flow. Static screens or a concept deck are not enough.
- A model, voice, language, and transport chosen for the use case.
- A visible and meaningful Qdrant moment. A clear, very clear edge of Rime over other popular voice models.
- Good handling of at least one difficult voice problem, such as latency, interruptions, pronunciation, delivery consistency, multilingual routing, or evaluation.
- Clear behavior when the system hits an error, a delay, an unsupported request, or a low confidence result.
- For agent products, a real action or safe handoff. For builder tools, a repeatable diagnosis or measurable improvement.
- A short forward-looking note on what model-internal continual learning could change if future BDH systems expose it publicly. Keep this clearly separated from what the public BDH repository can do today.

10 hour flight plan

REQUIREMENT	WHAT GOOD LOOKS LIKE
SCOPE	One user, one narrow problem, and one complete flow.
RIME	One carefully chosen model and voice, tested on the exact path used in the demo.
QDRANT	One meaningful collection and one visible retrieval, memory, routing, or evaluation moment.
HARD PROBLEM	One production difficulty handled deliberately, such as latency, interruption, pronunciation, multilingual routing, recovery, or evaluation.
PROOF	One action, handoff, recovery, or developer insight that can be demonstrated live.
POLISH	A reliable demo and a clean README. A hosted experience is optional unless the interface is central to the project.

RECOMMENDED 10 HOUR BUILD SEQUENCE

- Hour 1: Generate Rime audio and validate the exact voice, model, language, and transport.
- Hour 2: Choose one Rime mission route, one user, one narrow problem, and one failure to handle.
- Hours 3 to 6: Build the complete voice flow before adding polish.
- Hours 7 to 8: Add the meaningful Qdrant role and the action, handoff, or evaluation proof.
- Hour 9: Test interruptions, names, numbers, unsupported inputs, and the chosen failure state.
- Hour 10: Record the strongest moment, publish GitHub, and verify setup from a clean start.

Possible Mission plan

Build the same core experience first. Then deepen it in at least two areas.

- Persistent memory with a retention policy, user isolation, correction handling, and deletion behavior.
- Continuity across phone, browser voice, WhatsApp, email, or another channel, with one shared task state.
- Hybrid retrieval for semantic intent and exact terms such as account types, error codes, names, addresses, and policy numbers.
- Natural interruption handling that records what the user actually heard and updates the conversation state correctly.
- Tool execution with confirmations, retries, call transfer, and a human handoff.
- CRM or API integration, webhook events, post call workflows, or campaign analytics.
- Multilingual routing and code switching evaluation using real target language text.
- End to end latency measurement across recognition, reasoning, buffering, network, framework, and playback.
- A reusable voice agent evaluation or regression testing tool.
- A small test set with task success, retrieval quality, latency, recovery, and human voice ratings.

Voice track guardrails

REQUIREMENT	WHAT GOOD LOOKS LIKE
KEEP RESPONSES SHORT	Voice is sequential. Break long content into natural turns so users can interrupt and the first useful audio arrives quickly.
TEST THE EXACT PATH	HTTP, WebSockets, LiveKit, Pipecat, Vapi, Daily, Weya AI, and other integrations may behave differently. Test the route used in your final demo.
MEASURE USER LATENCY	Include speech recognition, language model generation, sentence buffering, network time, framework behavior, and playback. Do not report only language model latency.
TRACK WHAT WAS HEARD	When a user interrupts, record what audio actually played. Update the conversation state from what the user heard, not from the full text that the system intended to say.
PRONUNCIATION MATTERS	Test names, numbers, addresses, product terms, and domain vocabulary early. Coda takes delivery cues from wording and punctuation and does not support SSML or emotion tags.
MEMORY NEEDS BOUNDARIES	Store only what helps the task. Show how users are isolated and how a wrong, risky, or outdated memory can be corrected or removed.
SECURITY	Keep every credential on the server. Do not put real personal information in prompts, logs, screenshots, recordings, repositories, or support channels.
	QDRANT IS A CORE REQUIREMENT A decorative collection or a retrieval call that does not affect the product is not enough. The demo should make Qdrant's contribution visible.

Voice track judging

REQUIREMENT	WHAT GOOD LOOKS LIKE
30 POINTS: PROBLEM AND USEFULNESS	Does the project solve a clear problem for a defined user, and is voice the right interface?
25 POINTS: VOICE EXPERIENCE	Is the speech natural, clear, appropriate, consistent, responsive, and well integrated into the interaction?
20 POINTS: USE OF RIME	Does the project use Rime for essential speech, with a model, voice, language, and transport that suit the use case?
15 POINTS: TECHNICAL EXECUTION	Does the working flow use Qdrant meaningfully and handle the selected technical difficulty, agent workflow, errors, and recovery reliably?
10 POINTS: DEMO	Does the demonstration clearly show the user problem, the working product, the role of voice, and the strongest proof?



TRACK 02

DragonForge

Understand the Hatchling. Build for what comes next.

DragonForge is not limited to the [public BDH repository](#). Some of the most interesting properties of BDH emerge only in substantially trained systems and cannot reasonably be recreated during a student hackathon.

You are therefore welcome to work one level above the model itself: build benchmarks, evaluation tools, explainers, visualizations, reproducibility studies, or controlled experiments around the research questions BDH is designed to address. Use models available today where appropriate, and design your contribution so future architectures, including larger BDH models when publicly accessible, could be evaluated through the same framework.

THE DRAGONFORGE QUESTION

Do not ask “How do I make the public toy model perform a frontier capability?” Ask “What can I build today that would help us understand, evaluate, or communicate that capability when the models arrive?”

The public repo is one surface, not the boundary

WORK DIRECTLY WITH PUBLIC BDH	WORK ONE LEVEL ABOVE THE MODEL
• Instrument bdh.py and inspect real activations, RoPE, sparsity, causality, tensor shapes, or repeated levels. • Build a Manim (library used by 3Blue1Brown YouTube videos), web, Three.js, notebook, or other explainer that maps an established BDH mechanism to the paper and code. • Run small controlled experiments that are actually feasible with the public implementation. • Audit, reproduce, or improve an earlier community BDH project.	• Build benchmark and evaluation tooling around long horizon reasoning, continual learning, latent reasoning, memory, or interpretability. • Use models available today to create comparisons, failure analyses, diagnostic views, model adapters, or reproducibility studies. • Explain established BDH results such as monosemantic synapses or sparse activity with much better pedagogy and evidence. • Design the artifact so a larger BDH model could later enter the same framework without changing the question or metric.

BDH in five minutes

TERM	PLAIN ENGLISH MEANING
TOKEN	A small input unit. In the public trainer, text is represented as raw bytes.
NEURON COORDINATE	One position in the model's very large activation vector. These are mathematical units, not complete biological cells.
RELU AND SPARSITY	ReLU turns negative values into zero. You can measure how many coordinates are active. The public code does not promise a fixed sparsity percentage.
ROPE	Rotary Position Embedding. It rotates pairs of query and key coordinates according to token position, like clock hands moving at different speeds.
ATTENTION	The current activation pattern is compared with earlier positions. The public implementation builds the full causal token by token score matrix.
GATING	Retrieved information is expanded back into the large neuron space and multiplied with the neurons active now.
REPEATED LEVELS	The same main parameter matrices are reused across levels. The internal representation still changes at each level.
SYNAPTIC MEMORY	The paper gives BDH attention a synaptic memory interpretation. In bdh.py this state is not exposed as a persistent object between calls.

You must go through this BDH explainer before kicking off your building journey. That would help you be on the right track with the right foundations:

- <https://pathway.com/research/bdh-explainer>
- https://www.instagram.com/reel/DaK-dK_CW_c

10 hour mission: make one useful contribution

In ten hours, do not spend the event trying to train a large model. Pick one important BDH idea or research question and make a concrete builder contribution now. You may work directly with the public code, use released results from existing benchmarks, compare models you can access today, audit an earlier community project, or build a technically exact explanation.

MANIM OR INTERACTIVE EXPLAINER Take one established BDH idea and make it technically exact and memorable: attention becoming synaptic memory, the currency synapse, sparse activity, graph versus GPU views, or latent reasoning. The deliverable is the explanation people would actually share.	BENCHMARK X RAY Take an existing benchmark such as BABILong or a continual learning or memory benchmark. Use released results or run a small feasible subset on models available today. Add a diagnostic view the original benchmark does not make easy.
PUBLIC BDH MICROSCOPE Instrument the public model and expose real tensors, sparsity, RoPE behavior, causal attention, repeated level dynamics, or tensor footprints. Build the observability layer another student wishes existed.	MONOSEMANTICITY METHOD EXPLORER Turn Pathway's established concept selective synapse result into an interactive or statistical teaching artifact. Show what was measured, what the controls were, and how a future model could be tested through the same method.
COMMUNITY REPO AUDIT Choose an earlier BDH build. Reproduce the strongest part, identify what is fragile or unsupported, and ship a cleaner version with better tests, measurements, or explanation.	RESEARCH QUESTION MAP Choose an important BDH direction, study how it is evaluated today, and build one reusable comparison or adapter. Finish with the exact experiment you would run if a larger BDH model became accessible.
	WHAT WINNING LOOKS LIKE IN 10 HOURS A concrete builder contribution that exists now. One clean piece of evidence. One GitHub story that stands on its own. And a short section explaining the exact question this artifact would let you test with a larger BDH model.

Possible Mission plan: build something the next researcher can use

Five days gives you room to build a reusable evaluation layer, a careful experiment, a research instrument, a stronger reproduction, or a definitive explainer. The project does not need to use `bdh.py` at all. It does need to stay anchored to a question that Pathway's BDH work makes important, and it should leave behind something another builder can actually use.

POST TRANSFORMER EVALUATION LAB

Build a model agnostic harness around one capability BDH is designed to address. Evaluate current models where feasible, add stronger diagnostics than a single leaderboard score, and keep the interface ready for future recurrent or Post Transformer models.

CONTINUAL LEARNING DIAGNOSTICS

Use existing continual learning or memory benchmarks rather than inventing a dataset by default. Analyze acquisition, retention, stale knowledge updates, interference, or stability versus plasticity on models you can access today. Define the same test you would later run on BDH.

LATENT REASONING LAB

Use an accessible looped or recurrent depth model, or a controlled toy system, to study internal iteration count, convergence, stopping criteria, or overthinking. Explain why BDH makes this question interesting and what you would compare once BDH access exists.

LONG HORIZON REASONING OBSERVATORY

Build around BABILong or another established long horizon benchmark. Add task difficulty curves, failure taxonomies, latency, memory, cost, or reasoning depth views. Make a future BDH model another adapter, not a new benchmark.

SYNAPSESCOPE OR INTERPRETABILITY TOOLKIT

Build around BDH's established monosemanticity result: a Manim or interactive explanation, a concept selectivity analysis toolkit, a statistical explorer, or a reusable visualization layer. Separate published findings from anything you measure yourself.

REPLICATION RESCUE OR DEFINITIVE EXPLAINER

Audit and strengthen an earlier community project, or build the reference explainer the ecosystem is missing. Add controls, multiple seeds where relevant, quantitative evaluation, code to paper mapping, and a README another researcher would trust.

Established BDH findings: build on them, do not hide them

Some BDH phenomena are easiest to see in substantially trained models. That is a reason to anchor to the published result and build useful tooling, evaluation, or explanation around it, not a reason to pretend the result does not exist.

BDH RESULT	WHAT PATHWAY HAS SHOWN	WHAT A STUDENT CAN CONTRIBUTE
Monosemantic synapses	Pathway's BDH research identifies concept selective synapses in trained models. The currency synapse responds to different surface forms and languages, and the paper validates its selectivity against contrast sentences.	Build a Manim or interactive explanation, a statistical experiment explorer, or a reusable concept selectivity toolkit. If you probe an accessible model yourself, label that measurement separately from the established BDH result.
Sparse, predictability linked activity	BDH research reports sparse positive activations and a controlled experiment in which higher level activity becomes quieter as an input becomes more predictable.	Build observability, reproduce a small controlled version where feasible, compare inputs or configurations, or create metrics and visualizations that can later be reused on substantially larger models.
	THE DRAGONFORGE RESEARCH CONTRACT Label every major claim as ESTABLISHED, MEASURED, or EXPLORATORY. Established means Pathway or the paper has already demonstrated it. Measured means your team directly observed it. Exploratory means it is your hypothesis, simulation, extension, or BDH inspired experiment. All three are welcome. Every DragonForge README must also include "If We Had Access to a Larger BDH Model": the exact experiment, comparator, metric, and result that would support or challenge your hypothesis.	

Community repos to inspect, learn from, then beat

These were built under earlier hackathon constraints. Treat them as launchpads, not authorities. Study the useful ideas, reproduce what you can, identify weak measurements or unclear assumptions, and aim to leave behind something more correct, more reproducible, more useful, or much better explained. You can and should aim to do better than the reference repositories.

PROJECT	LOOK AT IT FOR	HOW TO DO BETTER
Model Merging + Visual Explainer https://github.com/Kumarutkarsh9470/Hostel2688 Demo: https://youtu.be/MIw_ITRiuyt?t=25	Multi page interaction design, sparsity views, graph storytelling, and the ambition of a Transformer Explainer style BDH experience.	Keep only the strongest pages. Map every visual to public code or the paper. Separate paper results from toy repo behavior. Add real measurements where possible.
Game of Life: Interpretable Circuits https://github.com/Choco-princess/BDH_Nexus Demo: https://youtu.be/J0KXfoDiO3M	A controlled task with known rules and an attempt to connect behavior to internal units.	Verify the implementation first. Use several seeds and explicit controls. If a claimed one to one circuit does not reproduce, document that rather than forcing the story.
Legal Violation Classifier https://github.com/Deb044/LegalClassifier Demo: https://youtu.be/B8afc055Njs?t=345	How to wrap a BDH experiment in a concrete real world story and dashboard.	Audit the comparison and training protocol. Prefer a smaller, verifiable claim to an impressive benchmark with mismatched baselines.
BDH Attention in a Diffusion Model https://github.com/spandan11106/BDH-Explainer Demo: https://youtu.be/m2QtAjVr-Kg?t=421	Cross domain creativity, 3D visualization, and long form teaching.	Clearly label what is vanilla BDH, what is BDH inspired, and what is a new architectural modification. Prioritize technical fidelity over breadth.

DragonForge judging

WEIGHT	WHAT JUDGES SHOULD REWARD
30 POINTS: BUILDER CONTRIBUTION	Did the team leave behind something concrete and useful now: an evaluation layer, experiment, explainer, visualization, reproducibility study, tool, or improved implementation?
25 POINTS: TECHNICAL AND RESEARCH FIDELITY	Does the project represent BDH, the benchmark, and the research result correctly? Are established, measured, and exploratory claims clearly separated?
20 POINTS: EVIDENCE AND EVALUATION	Are conclusions supported by tests, measurements, controls, released benchmark results, reproducible analysis, or clearly cited published evidence?
15 POINTS: BDH RELEVANCE AND NEXT QUESTION	Does the project engage with a question BDH is designed to address, and does the team state a specific experiment they would run with larger BDH access?
10 POINTS: GITHUB AND COMMUNICATION	Is the repository easy to inspect, run, learn from, and share? Can the team explain the contribution without research theatre?
	CONTRIBUTION BEATS SCALE A technically exact Manim explainer, a benchmark diagnostic, a strong reproduction, or one small research experiment can beat a larger implementation. DragonForge rewards work another builder or researcher would genuinely want to use.

04

The mission log

How to publish work that others can trust and share

Required GitHub structure

REQUIREMENT	WHAT GOOD LOOKS LIKE
ONE SENTENCE CLAIM	State the strongest result in one precise sentence.
PROBLEM OR QUESTION	Explain who the user is, or what research question you investigated.
ARCHITECTURE OR EVALUATION FLOW	Include a simple diagram showing the important components, benchmark flow, experiment design, or data path.
HOW TO RUN	Provide exact setup commands and one short path to reproduce the central result.
PROOF	Include metrics, tests, charts, recordings, or controlled examples that support the claim.
TECHNOLOGY OR RESEARCH ANCHOR	For VoxForge, identify the Rime model, voice, language, transport, Qdrant role, Weya role if used, and other services. For DragonForge, identify the BDH research direction or published result that anchors the project. If you use bdh.py, include the commit and configuration. If you use benchmarks or other models, identify them precisely.
LIMITATIONS	Say what the project does not establish. This increases trust.
TEAM CONTRIBUTIONS	Credit each member and any community code or AI tools used.
DEMO AND SHOWCASE CLIP	Link the full demo and include one short clip or GIF that communicates the strongest proof.
IF WE HAD LARGER BDH ACCESS	DragonForge only: state the exact experiment you would run, what you would compare BDH against, which metric or visualization from this repository you would reuse, and what result would support or challenge your hypothesis.

A strong demonstration structure

1. First 20 seconds: Show the problem and the result. Do not begin with architecture slides.
2. Next 60 seconds: Show the strongest live proof, including the memory, action, recovery, experiment, or test.
3. Next 30 seconds: Explain the architecture and the role of the sponsor technologies.
4. Final 20 seconds: State the metric, limitation, and next step.

Questions to take to mentors

MENTOR	THE BEST QUESTIONS TO ASK
RIME	Does this voice fit the user and language? How should the text be written for the intended delivery? Which transport path should we validate first?
QDRANT	What should be stored as knowledge, user memory, episodic memory, or evaluation data? Which payload filters prevent leakage? Should we use hybrid retrieval?
WEYA AI	Where should the agent speak, act, confirm, retry, transfer, or stop? How would the workflow behave across calls, tools, CRM updates, webhooks, campaigns, analytics, and multiple channels?
PATHWAY	Which BDH result or research direction are we anchoring to? Is our present contribution useful even without model access? What would be the most informative comparison or experiment to run when a larger BDH model becomes accessible?

Recommended learning paths

REQUIREMENT	WHAT GOOD LOOKS LIKE
VOXFORGE, FIRST 30 MINUTES	Generate one Rime audio sample, create one Qdrant collection, and write the single user problem your voice experience will solve.
VOXFORGE, NEXT 60 MINUTES	Validate the complete speech path, design the memory schema, and decide the action and failure state before building the interface.
DRAGONFORGE, FIRST 30 MINUTES	Read the BDH primer and the official Pathway material most relevant to the direction you care about. Decide whether you are explaining an established result, evaluating a broader capability, or investigating an open question.
DRAGONFORGE, NEXT 60 MINUTES	Define what you can contribute now with public code, existing benchmarks, released results, current models, or community repos. Then write the exact question you would test with a larger BDH model and the metric you would keep unchanged.

Official resources

- [Rime documentation](#): Start with the quickstart, then check models, voices, languages, streaming, prompting, and the exact integration path used in your demo.
- [Rime voice agent reference](#): A working LiveKit based starting point with Rime Coda, turn taking, barge in, and voice casting. Treat it as scaffolding, not a solution to copy. Aim to improve the product idea, Qdrant role, reliability, measurement, and repository quality.
- [Weya AI](#): Production voice workflows across calls and other channels, with a strong focus on task completion and customer operations.
- [Weya AI documentation](#): Voice agents, knowledge bases, campaigns, custom tools, CRM and API integrations, transfers, webhooks, and analytics.
- [Qdrant agentic builders guide](#): Ideas for agent memory, retrieval, filtering, evaluation, and observability.
- [Qdrant hybrid search](#): Combine semantic meaning with exact words, names, codes, and identifiers.
- [Qdrant filtering](#): Use payload conditions to isolate users, workflows, languages, risk levels, and other structured constraints.
- [Public BDH repository](#): The baseline BDH GPU implementation and toy training script.
- [BDH Explainer](#): The recommended conceptual learning path. Scheduled to be live from August 11.
- [The Equations of Reasoning](#): A plain language and mathematical guide to local BDH dynamics.
- [Dragon Hatchling paper](#): The original research paper. Read only the sections relevant to your project.
- [Transformer Explainer](#): A benchmark for clear interactive architecture education.
- [LLM Visualization](#): A reference for showing model operations spatially and interactively.

05

Final transmission

What should exist when the forge cools

VOXFORGE

Build a voice experience people would genuinely prefer to use because speech is essential, the system handles a real production difficulty, Qdrant changes the outcome in a meaningful way, and the experience reaches a useful result.

DRAGONFORGE

Make a builder contribution around the questions BDH is opening. Explain an established result better, build the evaluation layer a future BDH model could enter, strengthen a reproduction, create research tooling, or investigate one small open question. What you build today must stand on its own; BDH gives it direction and a sharper next experiment.

The StarForge standard

- One clear problem or research question.
- One working artifact that another person can run.
- One trustworthy proof that supports the central claim.
- One public GitHub repository that another builder is excited to discover.

Build the present. Explore the frontier. Leave a signal worth following.

May the force be with you!

STARFORGE 2026



BUILD THE PRESENT. EXPLORE THE FRONTIER. LEAVE A SIGNAL WORTH FOLLOWING.

THE STARFORGE STANDARD

One clear problem.

One working artifact.

One trustworthy proof.

One public repository another builder is excited to discover.



VOXFORGE

DRAGONFORGE

PUBLIC GITHUB REQUIRED • BUILD QUICKLY • VERIFY WHAT YOU SHIP